VPN Wireguard

Cel zadania:

Celem zadania jest skonfigurowanie meshowej sieci VPN za pomocą WireGuarda, dla trzech maszyn na serwerze Proxmox, zabezpieczenie dostępu do usługi SSH firewallem oraz skonfigurowanie routingu do dodatkowej podsieci przez VPN.

1. Zestawienie sieci:

-Wykorzystam swój serwer Proxmox, na którym postawię 3 maszyny wirtualne w tej samej podsieci

-Na VM-kach zainstaluję Debian Minimal

-Ustawiam sieć i sprawdzam czy maszyny wirtualne widzą się w sieci

-Na każdej maszynie zainstaluję WireGuard

-Skonfiguruję WireGuard tak, żeby stworzyć sieć typu mesh

-Sprawdzę komunikację przez VPN i pokażę, że urządzenia faktycznie komunikują się przez VPN, a nie przez sieć lokalną.

2. Firewall:

-Uruchomię na jednym hoście usługę SSH, spróbuję połączyć się z innego hosta z usługą zarówno przez LAN, jak i przez VPN

-Dodam regułę do firewalla, która odrzuca ruch inny niż z VPNa

-Ponowię test połączenia z usługa przez LAN i VPN po dodaniu reguły firewalla

3. Routing przez VPN:

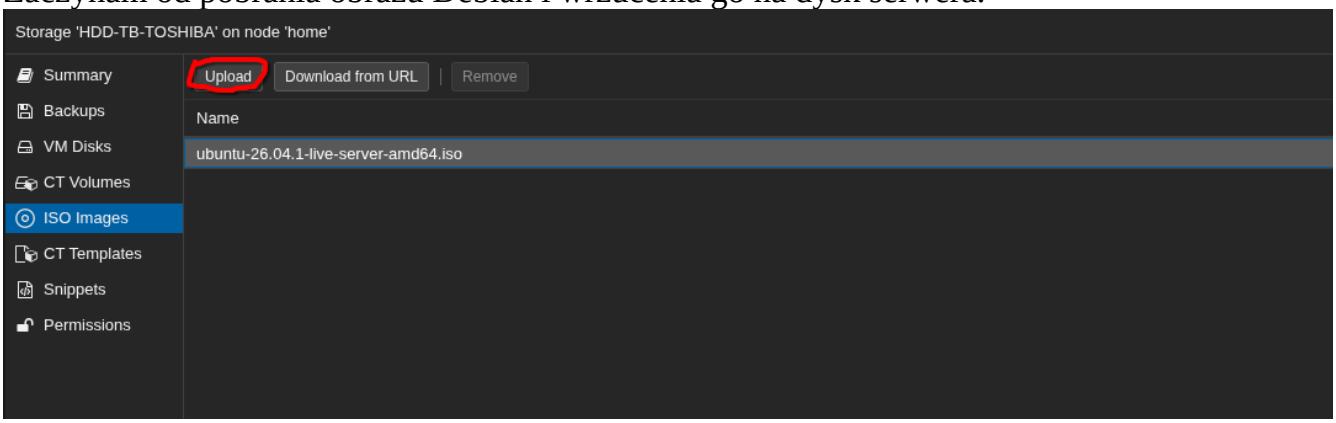
-Wybiorę kolejną podsieć i ustawię jednego hosta w tryb routera. Tylko ten host będzie podłączony do tej podsieci.

-Dodam i skonfiguruję routing przez VPN tak, żeby pozostałe hosty mogły komunikować się z nową podsiecią.

-Udowodnię, że ruch z pozostałych hostów do nowej podsieci jest przesyłany przez VPN.

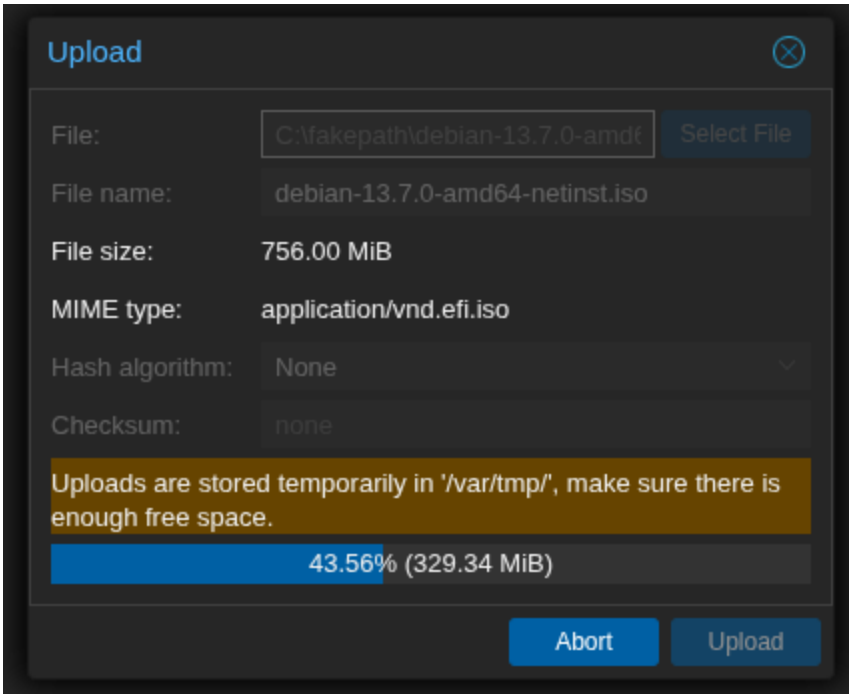
1. Zestawienie sieci

Stworzenie 3 maszyn na serwerze Proxmox.
Zaczynam od pobrania obrazu Debian i wrzucenia go na dysk serwera.

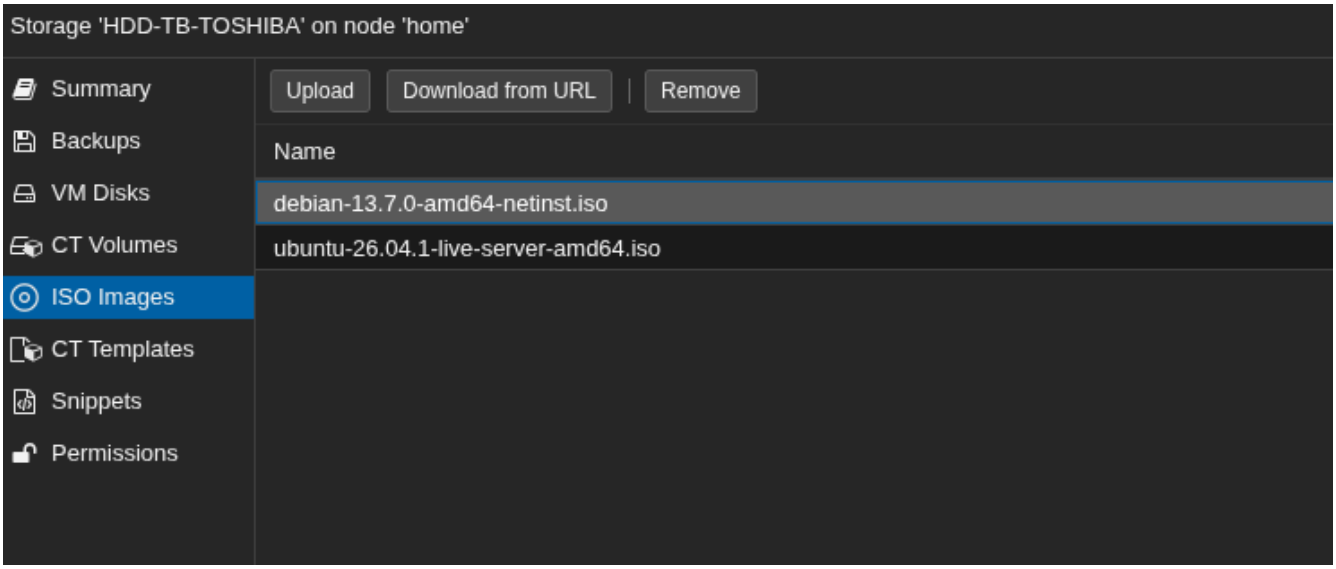


Dodajemy obraz poprzez Upload w zakładce ISO Images na naszym dysku.

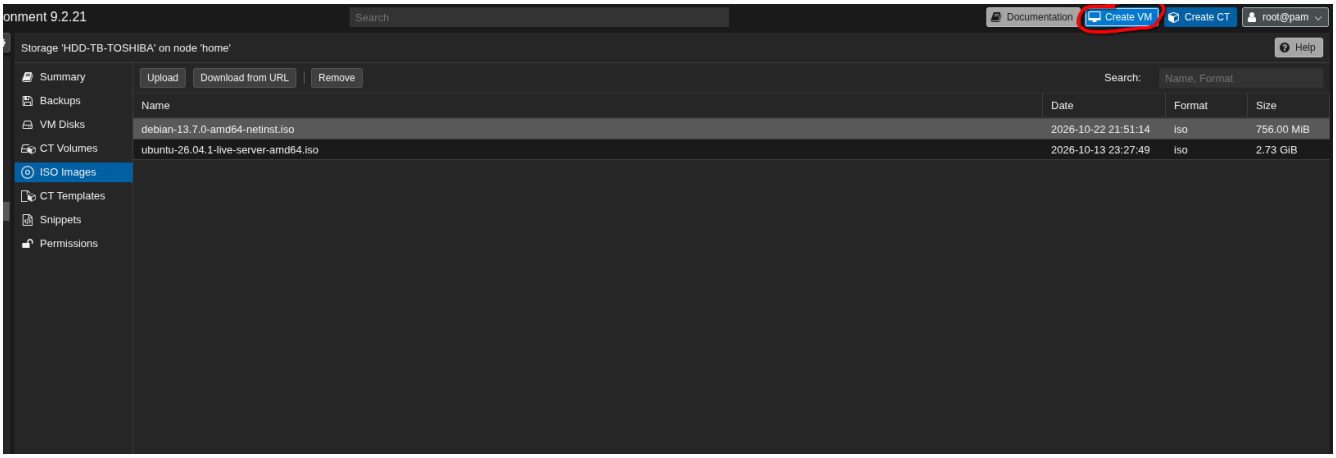
Wybieramy obraz oraz go uploadujemy



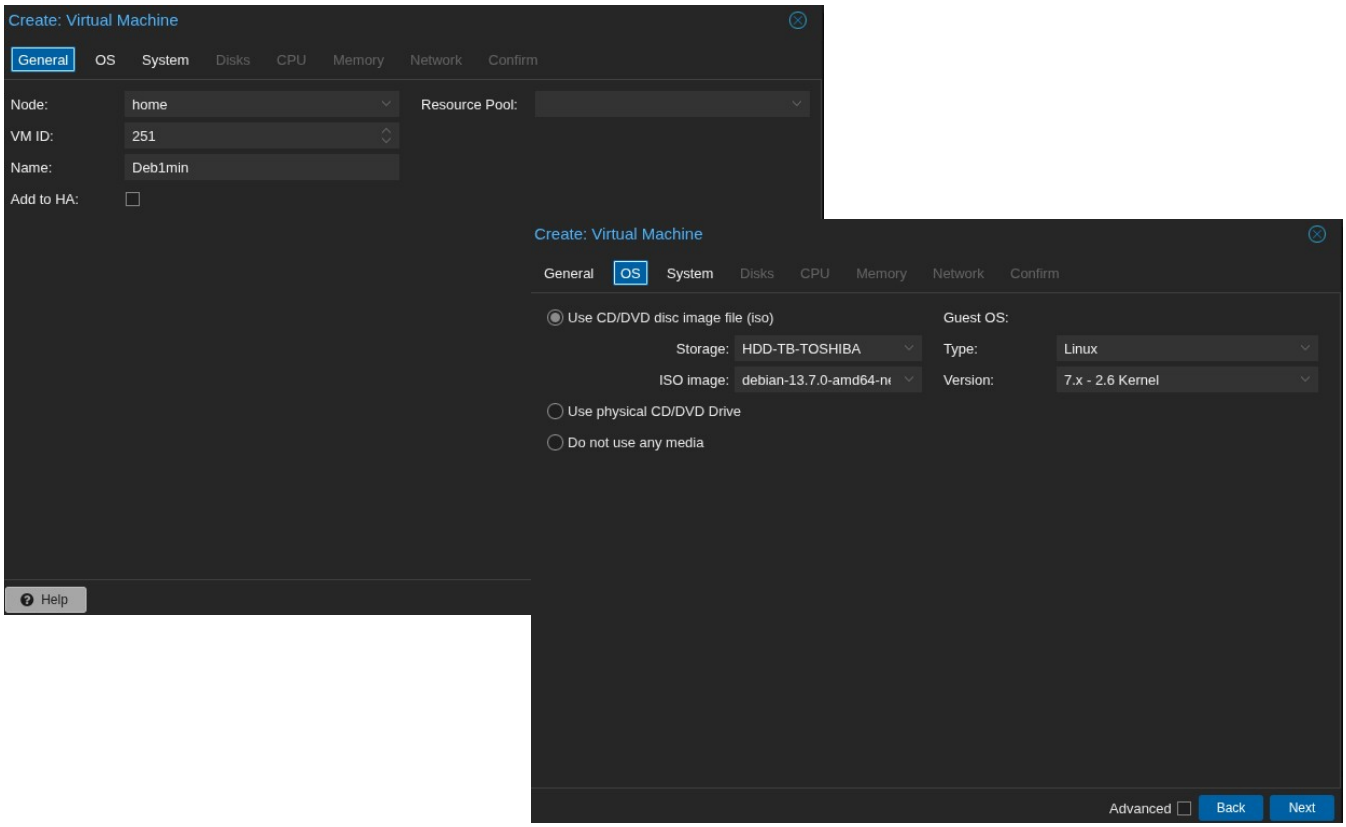
Mamy już nasz obraz na dysku



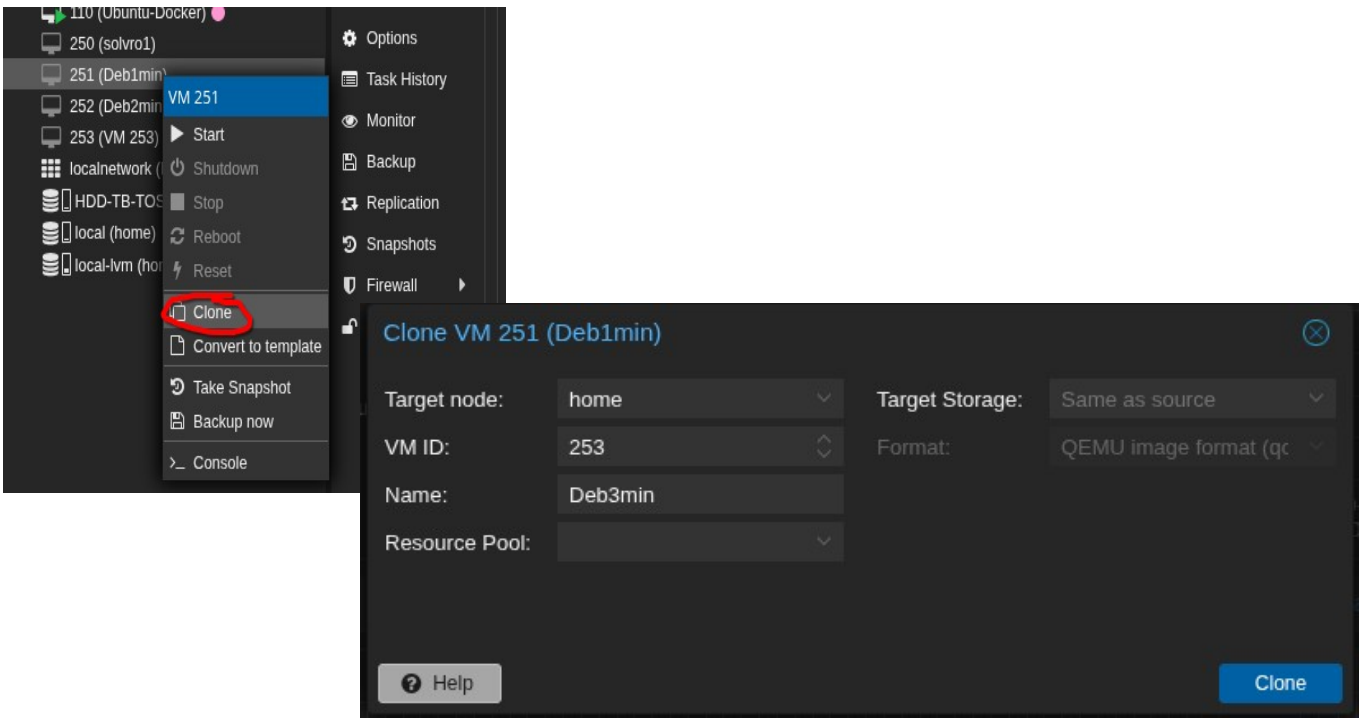
Teraz przejdziemy do utworzenia maszyn wirtualnych, aby to zrobić klikamy “Create VM”



Wpisujemy nazwę maszyny wirtualnej oraz ID. Wybieramy również obraz z którego ma powstać VM-ka i przypisujemy zasoby wedle uznania.



Dokonujemy klonowania aby uzyskać startowo te same wirtualne maszyny.



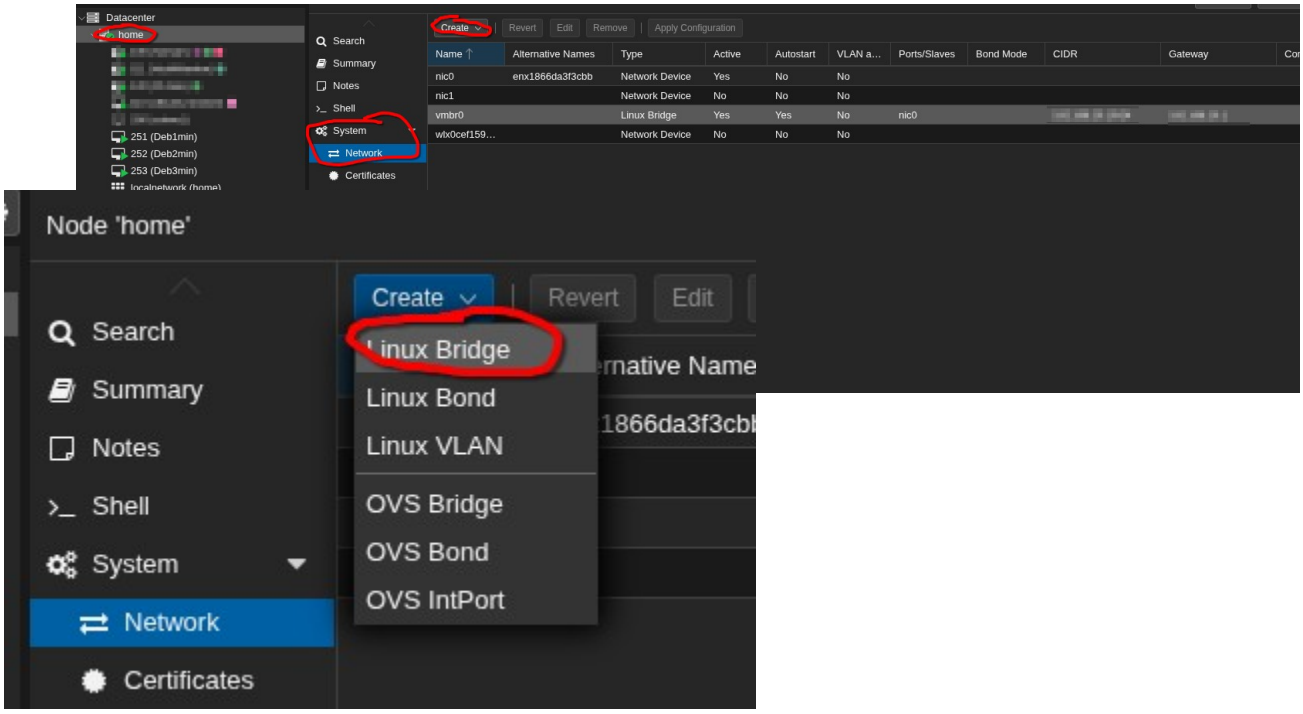
Teraz włączamy maszyny i dokonujemy instalacji Debian minimal.

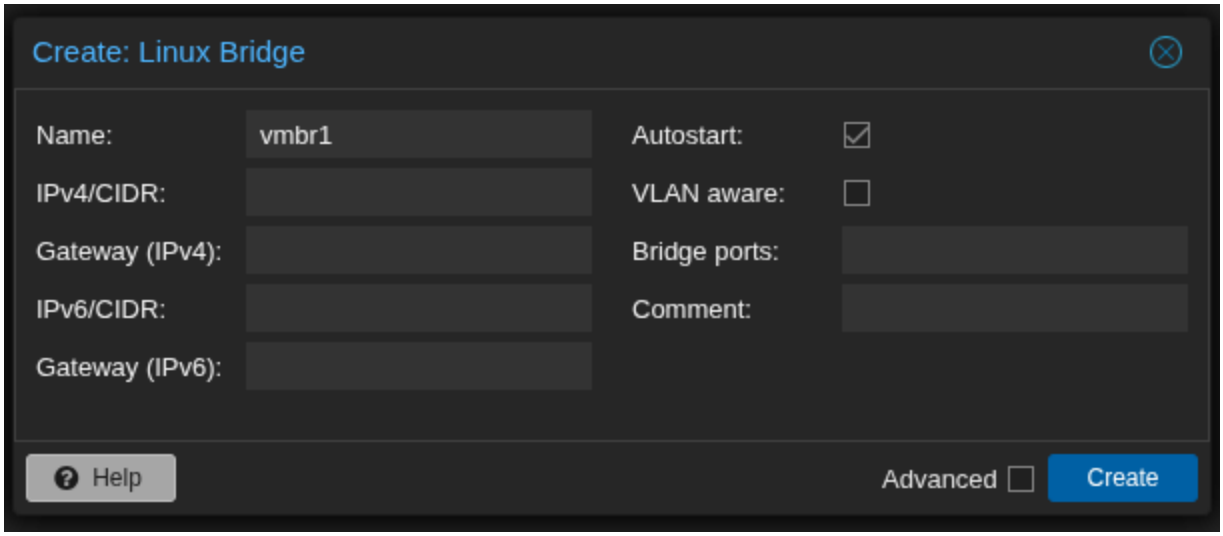
Aby dokonać instalacji minimalnej odznaczamy wszystko z tej listy poza SSH server który wykorzystamy potem do połączenia SSH oraz standard system utilities.



Mamy gotowe 3 maszyny wirtualne z Debianem minimal, teraz zrobimy bridgea w Proxmoxie aby dodać nasze VM-ki do innej podsieci niż mój serwer.

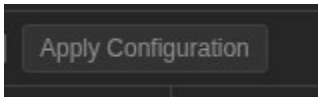
Dodajemy nowy bridge w naszym nodzie Proxmox.



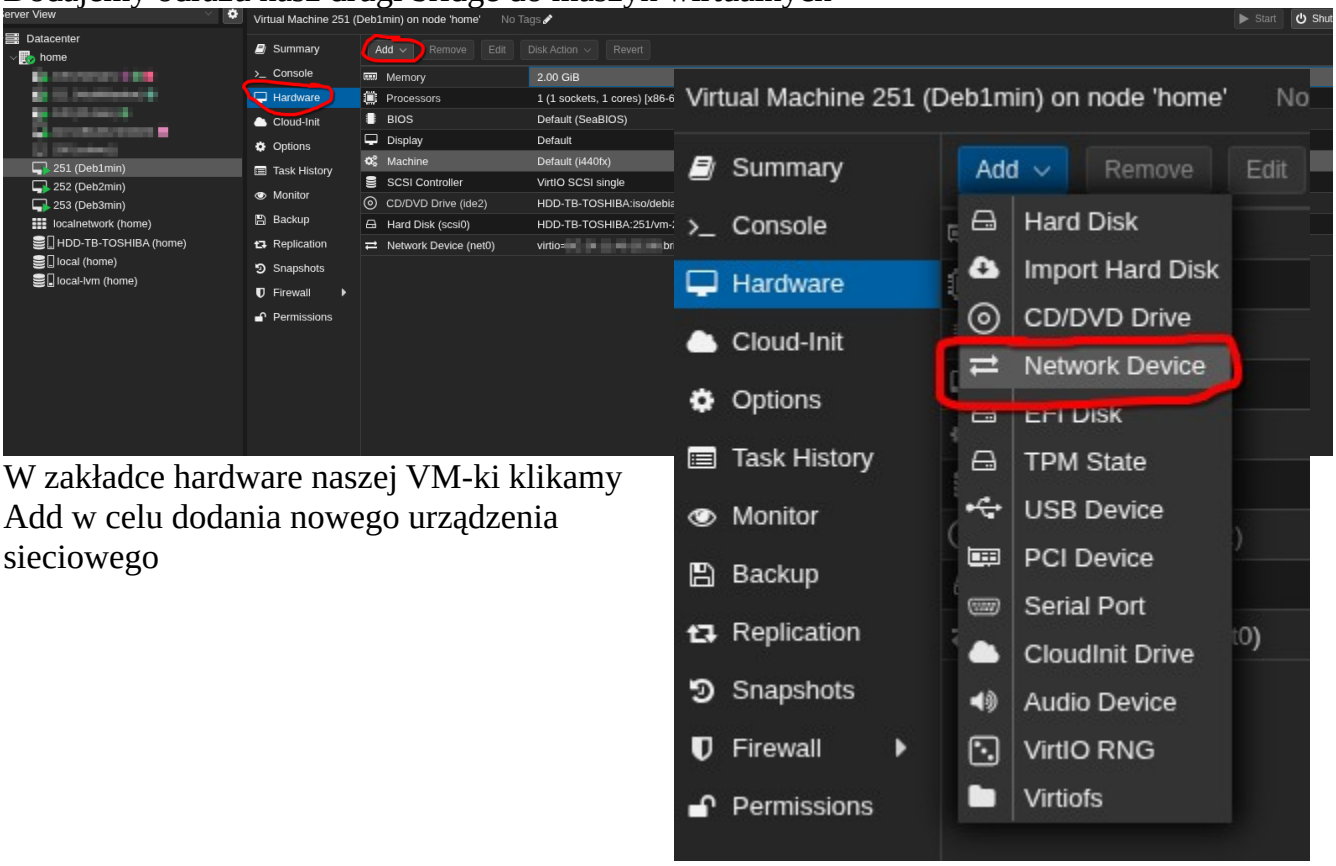


Zostawiamy puste pola przy IPv4/CIDR oraz Gateway, ponieważ nadamy w VM-kach adresy ręcznie oraz każda VM-ka będzie miała dwie kart sieciowe vmbr0 aby mieć dostęp do internetu i vmbr1 pod nasze zadanie.

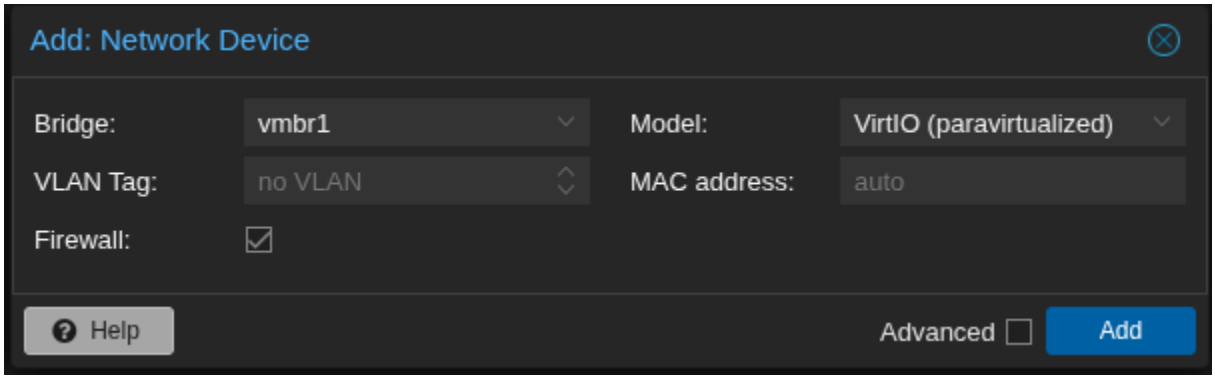
UWAGA! trzeba zatwierdzić konfigurację po dodaniu nowego bridgea.



Dodajemy odrazu nasz drugi bridge do maszyn wirtualnych



W zakładce hardware naszej VM-ki klikamy Add w celu dodania nowego urządzenia sieciowego



Wybieramy nasz bridge oraz zostawiamy VirtlO(paravirtualized)

Jak widać mamy dwa interfejsy sieciowe. Ens18 to interfejs vmbr0 z którego mamy dostęp do internetu, natomiast Ens19 to interfejs który posłuży nam do wykonania zadania.

```
deb1@debian1:~$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens18            UP             192.168.[redacted]/24 fe80::2619:[redacted]/64
ens19            DOWN
deb1@debian1:~$
```

Sprawdzamy czy internet działa.

```
deb1@debian1:~$ ping -c 3 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=53 time=9.71 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=53 time=9.41 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=53 time=8.95 ms

--- 1.1.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 8.949/9.355/9.710/0.312 ms
deb1@debian1:~$
```

Jak widać pingi przechodzą, więc mamy dostęp do internetu.

Teraz ustawiamy adresy IP na naszych maszynach.

```
deb1@debian1:~$ su
Password:
root@debian1:/home/deb1# ip addr add 192.168.100.11/24 dev ens19
root@debian1:/home/deb1# ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens18            UP             [redacted]/24 fe80::2619:[redacted]/64
ens19            DOWN           192.168.100.11/24
root@debian1:/home/deb1# ip link set ens19 up
root@debian1:/home/deb1# ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens18            UP             [redacted]/24 fe80::2619:[redacted]/64
ens19            UP             192.168.100.11/24 fe80::be24:11ff:fe0f:926d/64
root@debian1:/home/deb1#
```

Dodajemy nasz adres sieci z maską do interfejsu ens19 oraz włączamy go.
Adresacja maszyn:
VM1:“Deb1min”-->192.168.100.11/24
VM2:“Deb2min”-->192.168.100.12/24
VM3:“Deb3min”-->192.168.100.13/24

Teraz sprawdzamy czy maszyny się widzą w sieci LAN.
Ping VM1-->VM2

```
deb1@debian1:~$ ping -c 3 192.168.100.12
PING 192.168.100.12 (192.168.100.12) 56(84) bytes of data.
64 bytes from 192.168.100.12: icmp_seq=1 ttl=64 time=0.468 ms
64 bytes from 192.168.100.12: icmp_seq=2 ttl=64 time=0.523 ms
64 bytes from 192.168.100.12: icmp_seq=3 ttl=64 time=0.336 ms

--- 192.168.100.12 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 0.336/0.442/0.523/0.078 ms
```

Ping VM1-->VM3

```
deb1@debian1:~$ ping -c 3 192.168.100.13
PING 192.168.100.13 (192.168.100.13) 56(84) bytes of data.
64 bytes from 192.168.100.13: icmp_seq=1 ttl=64 time=0.454 ms
64 bytes from 192.168.100.13: icmp_seq=2 ttl=64 time=0.360 ms
64 bytes from 192.168.100.13: icmp_seq=3 ttl=64 time=0.497 ms

--- 192.168.100.13 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.360/0.437/0.497/0.057 ms
```

Ping VM2-->VM3

```
debian2min@deb2min:~$ ping -c 3 192.168.100.13
PING 192.168.100.13 (192.168.100.13) 56(84) bytes of data.
64 bytes from 192.168.100.13: icmp_seq=1 ttl=64 time=0.767 ms
64 bytes from 192.168.100.13: icmp_seq=2 ttl=64 time=0.525 ms
64 bytes from 192.168.100.13: icmp_seq=3 ttl=64 time=0.361 ms

--- 192.168.100.13 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2026ms
rtt min/avg/max/mdev = 0.361/0.551/0.767/0.166 ms
debian2min@deb2min:~$
```

Ping VM3-->VM1

```
debian3min@deb3min:~$ ping -c 3 192.168.100.11
PING 192.168.100.11 (192.168.100.11) 56(84) bytes of data.
64 bytes from 192.168.100.11: icmp_seq=1 ttl=64 time=0.535 ms
64 bytes from 192.168.100.11: icmp_seq=2 ttl=64 time=0.515 ms
64 bytes from 192.168.100.11: icmp_seq=3 ttl=64 time=0.488 ms

--- 192.168.100.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2026ms
rtt min/avg/max/mdev = 0.488/0.512/0.535/0.019 ms
debian3min@deb3min:~$ _
```

Jak widać wszystkie pingi działają.

Teraz przechodzimy do instalacji Wireguarda na każdej maszynie

```
root@debian1:/home/deb1# apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://security.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:3 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:4 http://security.debian.org/debian-security trixie-security/main Sources [238 kB]
Get:5 http://security.debian.org/debian-security trixie-security/main amd64 Packages [270 kB]
Get:6 http://security.debian.org/debian-security trixie-security/main Translation-en [166 kB]
Fetched 765 kB in 1s (1,518 kB/s)
1 package can be upgraded. Run 'apt list --upgradable' to see it.
root@debian1:/home/deb1# apt install wireguard -y
Installing:
  wireguard

Installing dependencies:
  wireguard-tools

Suggested packages:
  openresolv  | resolvconf
```

Następnie potrzebujemy wygenerować klucz base64-encoded, publiczny i prywatny na każdej VM-ce osobno.
Klucze publiczne posłuża do weryfikacji połączenia oraz szyfrowania danych pomiędzy maszynami.

Klucz generujemy w ten sposób:

```
root@debian1:/home/deb1# umask 077
root@debian1:/home/deb1# wg genkey > /etc/wireguard/private.key
root@debian1:/home/deb1# wg pubkey < /etc/wireguard/private.key > /etc/wireguard/public.key
root@debian1:/home/deb1# cat /etc/wireguard/public.key
i2zOOFm6p5dDYmMU+UHRTc505[redacted]
root@debian1:/home/deb1#
```

1. Ustawiamy uprawnienia nowego pliku read-write-execute tylko dla właściciela.

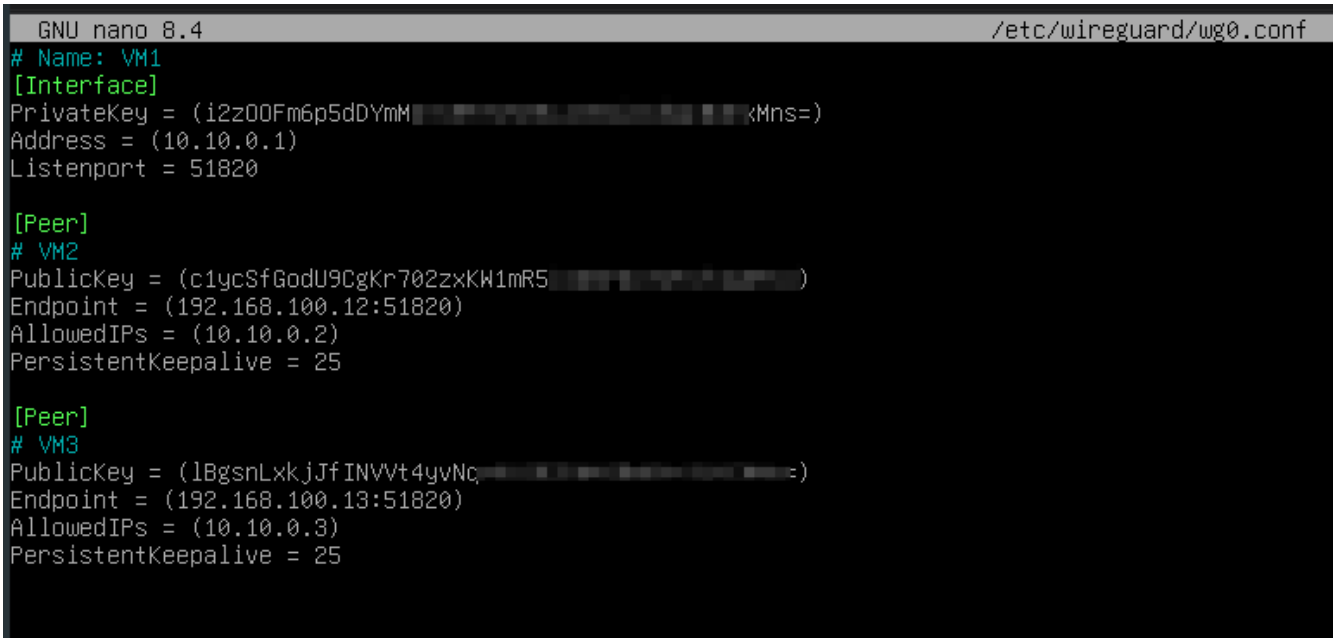
2. Generujemy klucz do pliku private.key z którego potem eksportujemy klucz publiczny do nowego pliku.

3.Odczytujemy klucz publiczny, posłuży nam on zaraz do konfiguracji.

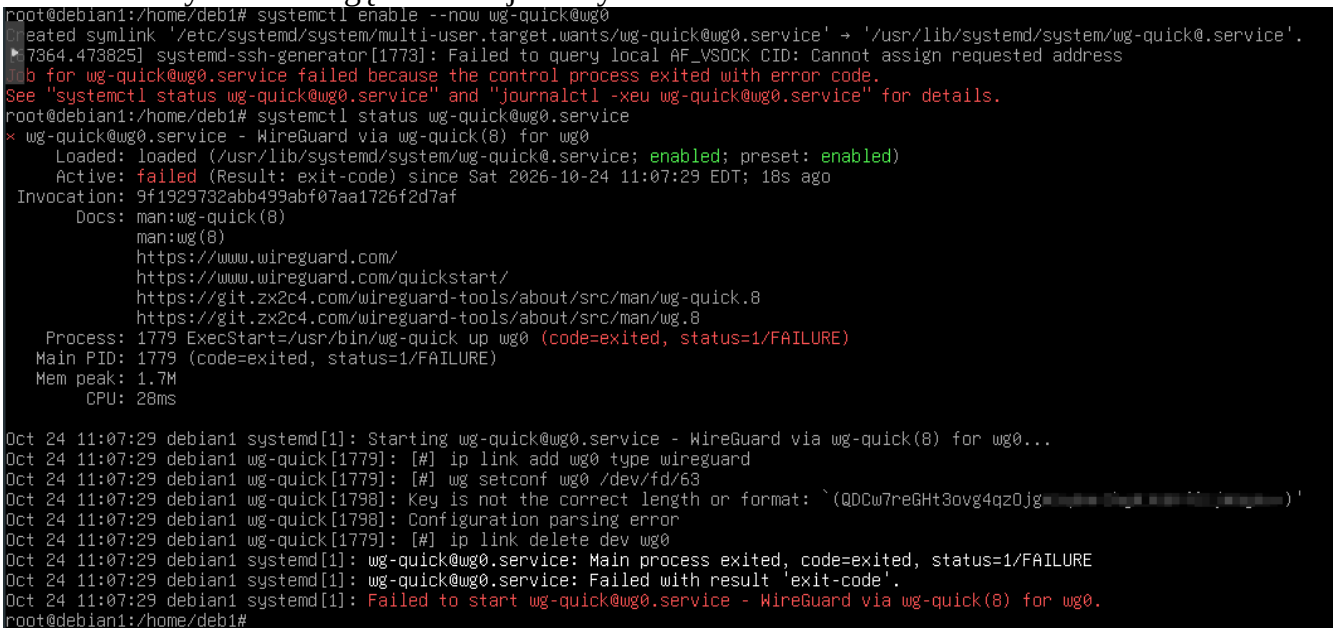
Tworzymy plik /etc/wireguard/wg0.conf na każdej maszynie i skonfigurujemy go tak aby stworzyć VPN mesha

Na każdej VM-ce robimy podobnie, ustawiamy interfejs własny w którym:
-dajemy nasz wygenerowany klucz prywatny (PrivateKey),
-adres VPN dla naszej maszyny(Address)
-port do nasłuchu(Listenport)

Następnie tworzymy węzły z VM 2 i VM3, a w nich:
-dodajemy klucze publiczne z danych maszyn(PublicKey),
-adres i port pod który mają docierać zaszyfrowane pakiety(Endpoint),
-dozwolone adresy IP które mogą się komunikować z naszą maszyną(AllowedIPs),
-wysyłanie pakietu co 25s aby podtrzymać nasze mapowanie sieciowe(PersistentKeepalive)
(standardowo po 30s te mapowanie wygasa)

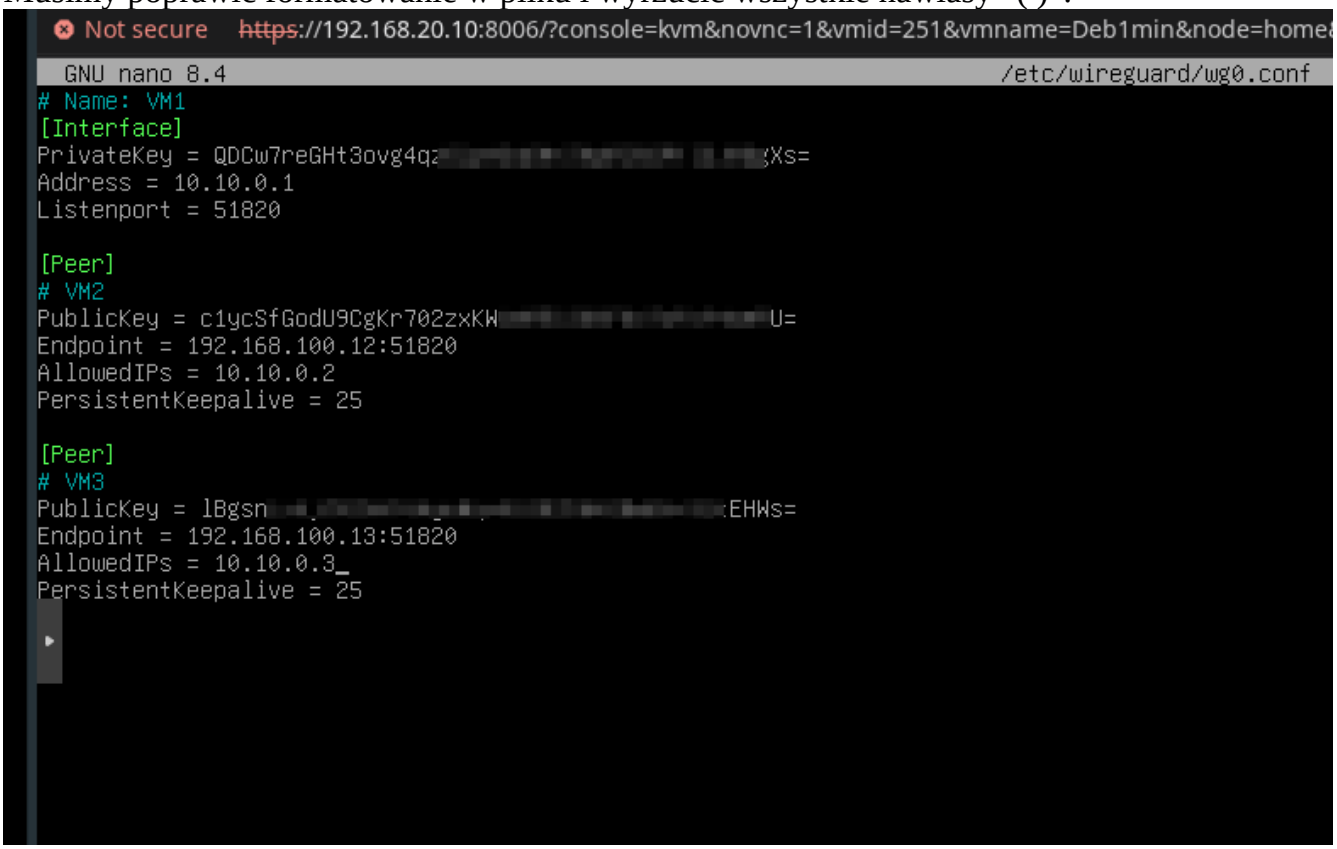


Uruchamiamy teraz usługę na każdej maszynie.



Niestety mamy błąd, z logów widać że przepisaliśmy źle klucz.

Musimy poprawić formatowanie w pliku i wyrzucić wszystkie nawiasy “( )”.



Poprawiamy pliki konfiguracyjne oraz sprawdzamy teraz czy nasza usługa wystartuje.

```
root@debian1:/home/deb1# systemctl restart wg-quick@wg0
root@debian1:/home/deb1# systemctl status wg-quick@wg0
● wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0
     Loaded: loaded (/usr/lib/systemd/system/wg-quick@.service; enabled; preset: enabled)
     Active: active (exited) since Sat 2026-10-24 11:16:37 EDT; 9s ago
 Invocation: ef5ad62c37074631b312e90071ecc713
       Docs: man:wg-quick(8)
             man:wg(8)
             https://www.wireguard.com/
             https://www.wireguard.com/quickstart/
             https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
             https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8
    Process: 1955 ExecStart=/usr/bin/wg-quick up wg0 (code=exited, status=0/SUCCESS)
   Main PID: 1955 (code=exited, status=0/SUCCESS)
   Mem peak: 2M
        CPU: 49ms

Oct 24 11:16:37 debian1 systemd[1]: Starting wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0...
Oct 24 11:16:37 debian1 wg-quick[1955]: [#] ip link add wg0 type wireguard
Oct 24 11:16:37 debian1 wg-quick[1955]: [#] wg setconf wg0 /dev/fd/63
Oct 24 11:16:37 debian1 wg-quick[1955]: [#] ip -4 address add 10.10.0.1 dev wg0
Oct 24 11:16:37 debian1 wg-quick[1955]: [#] ip link set mtu 1420 up dev wg0
Oct 24 11:16:37 debian1 wg-quick[1955]: [#] ip -4 route add 10.10.0.3/32 dev wg0
Oct 24 11:16:37 debian1 wg-quick[1955]: [#] ip -4 route add 10.10.0.2/32 dev wg0
Oct 24 11:16:37 debian1 systemd[1]: Finished wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0.
root@debian1:/home/deb1#
```

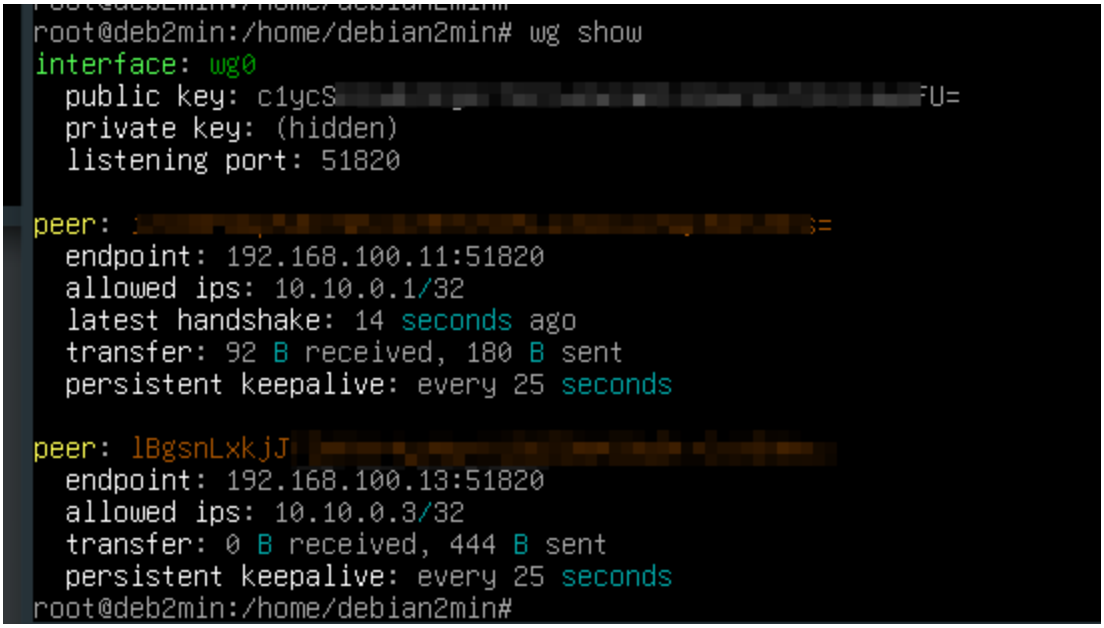
Restartujemy usługę i sprawdzamy jej status

```
root@debian1:/home/deb1# wg show
interface: wg0
  public key: i2zOOFm6p5[redacted]
  private key: (hidden)
  listening port: 51820

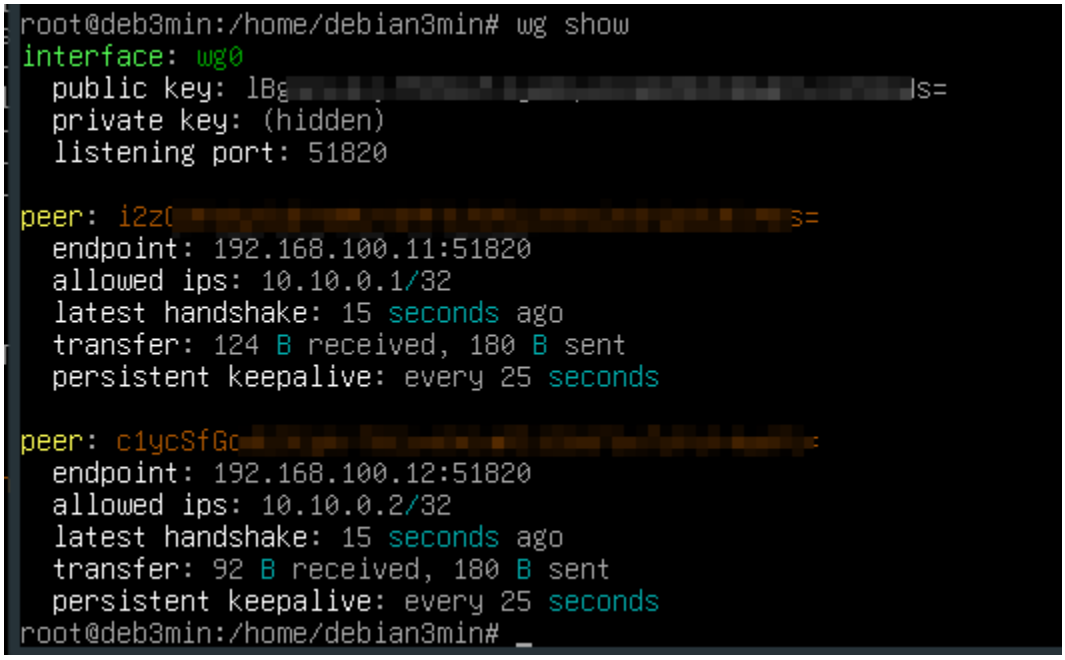
peer: c1ycS[redacted]f4weFU=
  endpoint: 192.168.100.12:51820
  allowed ips: 10.10.0.2/32
  transfer: 0 B received, 2.60 KiB sent
  persistent keepalive: every 25 seconds

peer: lBgsr[redacted]tEHWs=
  endpoint: 192.168.100.13:51820
  allowed ips: 10.10.0.3/32
  transfer: 0 B received, 2.60 KiB sent
  persistent keepalive: every 25 seconds
root@debian1:/home/deb1#
```

Nasza aktualna konfiguracja wygląda tak na VM1


root@deb2min:/home/debian2min# wg show
interface: wg0
public key: c1ycS
FU=
private key: (hidden)
listening port: 51820
peer:
endpoint: 192.168.100.11:51820
allowed ips: 10.10.0.1/32
latest handshake: 14 seconds ago
transfer: 92 B received, 180 B sent
persistent keepalive: every 25 seconds
peer: lBgsnLxkjJ
endpoint: 192.168.100.13:51820
allowed ips: 10.10.0.3/32
transfer: 0 B received, 444 B sent
persistent keepalive: every 25 seconds
root@deb2min:/home/debian2min#

VM2


root@deb3min:/home/debian3min# wg show
interface: wg0
public key: lBg
ls=
private key: (hidden)
listening port: 51820
peer: i2zC
s=
endpoint: 192.168.100.11:51820
allowed ips: 10.10.0.1/32
latest handshake: 15 seconds ago
transfer: 124 B received, 180 B sent
persistent keepalive: every 25 seconds
peer: c1ycSfGc
endpoint: 192.168.100.12:51820
allowed ips: 10.10.0.2/32
latest handshake: 15 seconds ago
transfer: 92 B received, 180 B sent
persistent keepalive: every 25 seconds
root@deb3min:/home/debian3min#

VM3

Teraz testujemy połączenie przez tunel.
VM1 do VM2 i VM3

```
root@debian1:/home/deb1# ping -c 3 10.10.0.2
PING 10.10.0.2 (10.10.0.2) 56(84) bytes of data.
64 bytes from 10.10.0.2: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 10.10.0.2: icmp_seq=2 ttl=64 time=0.574 ms
64 bytes from 10.10.0.2: icmp_seq=3 ttl=64 time=0.599 ms

--- 10.10.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.574/0.787/1.190/0.284 ms
root@debian1:/home/deb1# ping -c 3 10.10.0.3
PING 10.10.0.3 (10.10.0.3) 56(84) bytes of data.
64 bytes from 10.10.0.3: icmp_seq=1 ttl=64 time=0.584 ms
64 bytes from 10.10.0.3: icmp_seq=2 ttl=64 time=0.634 ms
64 bytes from 10.10.0.3: icmp_seq=3 ttl=64 time=0.594 ms

--- 10.10.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.584/0.604/0.634/0.021 ms
root@debian1:/home/deb1#
```

VM2 do VM1 i VM3

```
root@deb2min:/home/debian2min# ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=0.638 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=0.564 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=0.702 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014ms
rtt min/avg/max/mdev = 0.564/0.634/0.702/0.056 ms
root@deb2min:/home/debian2min# ping -c 3 10.10.0.3
PING 10.10.0.3 (10.10.0.3) 56(84) bytes of data.
64 bytes from 10.10.0.3: icmp_seq=1 ttl=64 time=0.722 ms
64 bytes from 10.10.0.3: icmp_seq=2 ttl=64 time=0.492 ms
64 bytes from 10.10.0.3: icmp_seq=3 ttl=64 time=0.579 ms

--- 10.10.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2017ms
rtt min/avg/max/mdev = 0.492/0.597/0.722/0.094 ms
root@deb2min:/home/debian2min#
```

VM3 do VM1 i VM2

```
root@deb3min:/home/debian3min# ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=1.21 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=0.680 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=0.640 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2023ms
rtt min/avg/max/mdev = 0.640/0.844/1.213/0.261 ms
root@deb3min:/home/debian3min# ping -c 3 10.10.0.2
PING 10.10.0.2 (10.10.0.2) 56(84) bytes of data.
64 bytes from 10.10.0.2: icmp_seq=1 ttl=64 time=1.41 ms
64 bytes from 10.10.0.2: icmp_seq=2 ttl=64 time=0.689 ms
64 bytes from 10.10.0.2: icmp_seq=3 ttl=64 time=0.608 ms

--- 10.10.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2028ms
rtt min/avg/max/mdev = 0.608/0.901/1.407/0.359 ms
root@deb3min:/home/debian3min#
```

Jak widac pingi działają, natomiast teraz sprawdzimy czy faktycznie przechodzą przez tunel.

```
root@debian1:/home/deb1# wg show
interface: wg0
  public key: i2zOOFm6p5dDYmM
  private key: (hidden)
  listening port: 51820

peer: c1ycS
  endpoint: 192.168.100.12:51820
  allowed ips: 10.10.0.2/32
  latest handshake: 1 minute, 7 seconds ago
  transfer: 2.04 KiB received, 10.21 KiB sent
  persistent keepalive: every 25 seconds

peer: lB                                    Ws=
  endpoint: 192.168.100.13:51820
  allowed ips: 10.10.0.3/32
  latest handshake: 1 minute, 9 seconds ago
  transfer: 1.26 KiB received, 13.49 KiB sent
  persistent keepalive: every 25 seconds
root@debian1:/home/deb1# _
```

Na VM1 jak widać, pingi przeszły przez tunel. Wskazuje na to latest handshake oraz transfer.

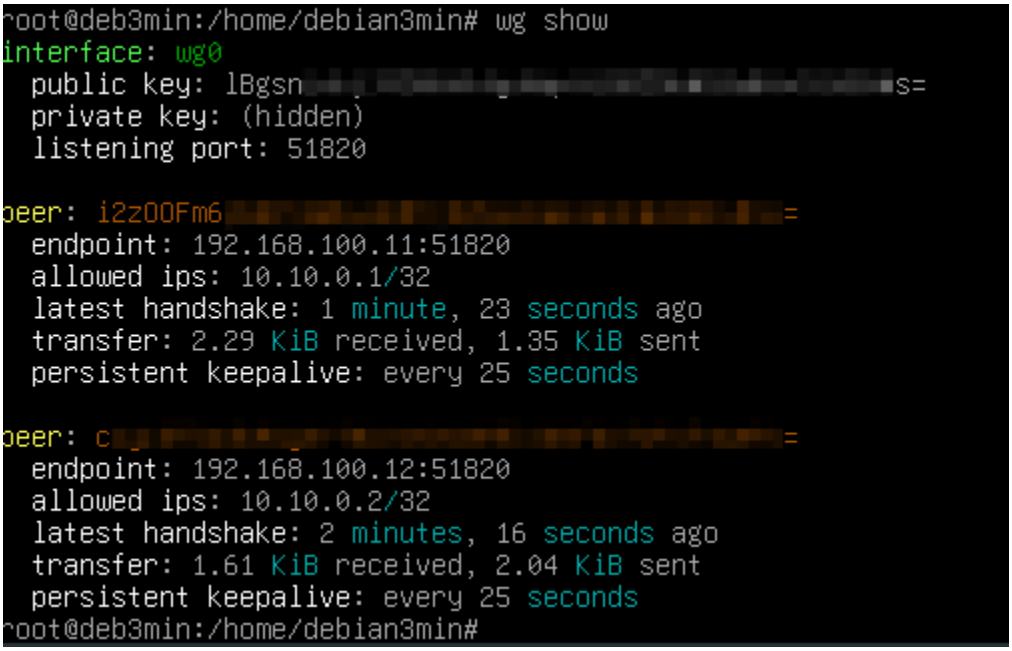
Sprawdzmy na pozostałych.

```
root@deb2min:/home/debian2min# wg show
interface: wg0
  public key: c1ycSf                                FU=
  private key: (hidden)
  listening port: 51820

peer: i2                                          s=
  endpoint: 192.168.100.11:51820
  allowed ips: 10.10.0.1/32
  latest handshake: 22 seconds ago
  transfer: 1.70 KiB received, 2.27 KiB sent
  persistent keepalive: every 25 seconds

peer: lBgsnLxk                                    s=
  endpoint: 192.168.100.13:51820
  allowed ips: 10.10.0.3/32
  latest handshake: 1 minute, 37 seconds ago
  transfer: 2.00 KiB received, 4.46 KiB sent
  persistent keepalive: every 25 seconds
root@deb2min:/home/debian2min#
```

VM2



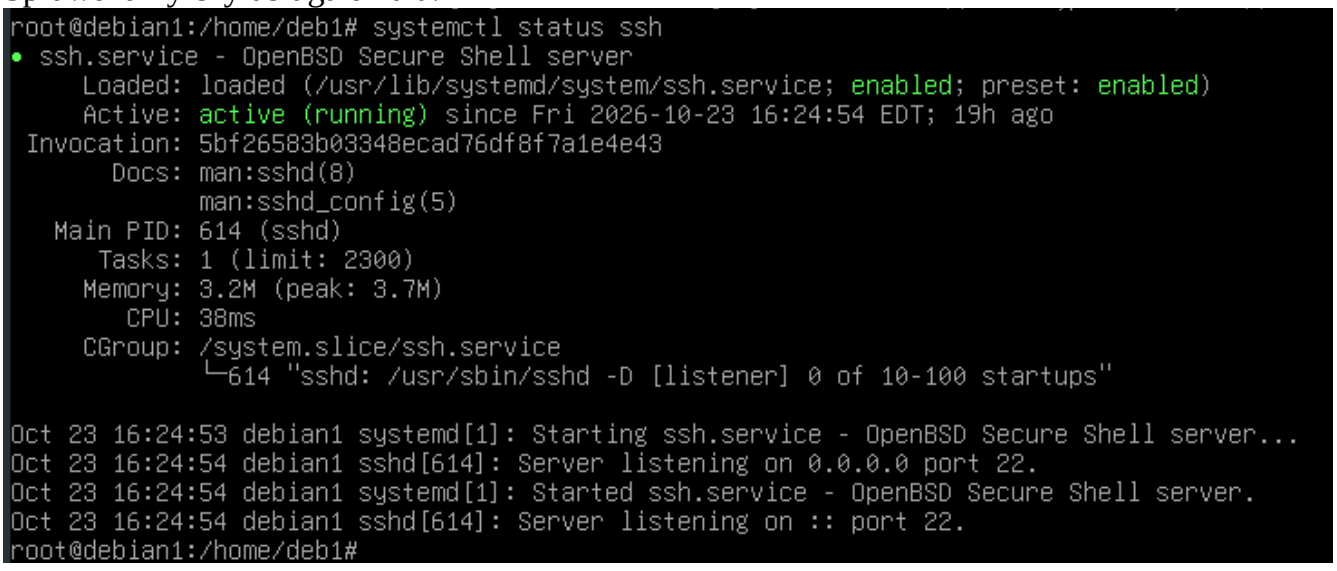
VM3

Na pozostałych maszynach równiez ruch przechodzi przez tunel.

2. Firewall

Teraz uruchamiamy usługę SSH na wybranym hoście, ja wybrałem VM1.

Sprawdzamy czy usługa działa.



Jak widać usługa działa oraz widzimy że nasłuchuje na wszystkich adresach IP(0.0.0.0) na porcie 22.

Łączymy się do SSH przez LAN i VPN.

```
root@deb2min:/home/debian2min# ssh deb1@192.168.100.11
The authenticity of host '192.168.100.11 (192.168.100.11)' can't be established.
ED25519 key fingerprint is SHA256:XTMrHK+E6y93apYccaJj97brbTIjgRmf0RAyJLaEXHw.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.100.11' (ED25519) to the list of known hosts.
deb1@192.168.100.11's password:
Linux debian1 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Oct 24 12:14:35 2026 from 10.10.0.2
deb1@debian1:~$
```

```
root@deb2min:/home/debian2min# ssh deb1@10.10.0.1
deb1@10.10.0.1's password:
Linux debian1 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
deb1@debian1:~$
```

Jak widać połączenie przez LAN i VPN działa.

Teraz przejdziemy do dodania reguły w firewallu poprzez iptables, która będzie odrzucać pakiety kierowane spoza naszego VPNa do naszej usługi.

Musiałem doinstalować iptables bo mój Debian minimal go nie posiadał.

```
root@debian1:/home/deb1#
root@debian1:/home/deb1# apt install iptables -y
```

Musiałem naprawić znajdowanie ścieżki do iptables, ponieważ nie znajdowało komendy.

```
root@debian1:/home/deb1# export PATH="$PATH:/usr/sbin:/sbin"
root@debian1:/home/deb1# iptables
iptables v1.8.11 (nf_tables): no command specified
Try `iptables -h' or 'iptables --help' for more information.
root@debian1:/home/deb1#
root@debian1:/home/deb1# _
```

Teraz sprawdzamy aktualne reguły.

```
root@debian1:/home/deb1# iptables -nvL
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
root@debian1:/home/deb1# _
```

Nie ma jeszcze żadnych reguł.

Teraz dodajemy regułę do iptables która zablokuje pakiety spoza VPN do usługi SSH.

```
root@debian1:/home/deb1# iptables -A INPUT -p tcp --dport 22 ! -i wg0 -j DROP
```

flaga -A dodaje regułę dotyczącą pakietów przychodzących (INPUT), protokół tcp, docelowy port 22, negacja(!) która mówi że spoza interfejsu wg0 (DROP) odrzuć pakiet.

Sprawdzamy czy nasza reguła się dodała.

```
root@debian1:/home/deb1# iptables -L INPUT -n -v --line-numbers
Chain INPUT (policy ACCEPT 9 packets, 776 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       tcp  --  !wg0   *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
root@debian1:/home/deb1#
```

Nasza reguła się dodała.

Teraz testujemy ponownie po dodaniu reguły w firewallu połączenie poprzez LAN i VPN.

```
root@deb2min:/home/debian2min# ssh deb1@192.168.100.11
^X^Z
[5]+  Stopped                 ssh deb1@192.168.100.11
root@deb2min:/home/debian2min# ssh deb1@10.10.0.1
deb1@10.10.0.1's password:
Linux debian1 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Oct 24 12:15:49 2026 from 192.168.100.12
deb1@debian1:~$
deb1@debian1:~$
deb1@debian1:~$ exit
logout
Connection to 10.10.0.1 closed.
root@deb2min:/home/debian2min#
  ot@deb2min:/home/debian2min#
  ot@deb2min:/home/debian2min#
```

Połączenie przez VPN działa a przez LAN nie.

```
root@debian1:/home/deb1# iptables -L INPUT -n -v --line-numbers
Chain INPUT (policy ACCEPT 447 packets, 71820 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1       40  6560 DROP       tcp  --  !wg0   *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
root@debian1:/home/deb1#
```

Tutaj w regule widzimy kolumne pkts która zlicza pakiety na których zastosowano regułę, więc nasza reguła działa ponieważ licznik się zwiększa oraz połączenie jest niemożliwe spoza VPNa, dokładnie to chcieliśmy osiągnąć.

Ta sama sytuacja ma miejsce gdy chcemy się połączyć z innej maszyny.

```
root@deb3min:/home/debian3min# ssh deb1@192.168.100.11
^Z
[1]+  Stopped                 ssh deb1@192.168.100.11
root@deb3min:/home/debian3min# ssh deb1@10.10.0.1
The authenticity of host '10.10.0.1 (10.10.0.1)' can't be established.
ED25519 key fingerprint is SHA256:XTMrHK+E6y93apYccaJj97brbTIjgRmf0RAyJLaEXWw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.0.1' (ED25519) to the list of known hosts.
deb1@10.10.0.1's password:
Linux debian1 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Oct 24 13:10:04 2026 from 10.10.0.2
deb1@debian1:~$ exit
logout
Connection to 10.10.0.1 closed.
```

```
root@debian1:/home/deb1# iptables -L INPUT -n -v --line-numbers
Chain INPUT (policy ACCEPT 571 packets, 93845 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1       49  7100 DROP       tcp  --  !wg0   *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
root@debian1:/home/deb1#
```

3. Routing przez VPN

Teraz dodamy adres nowej podsieci do interfejsu “lo” oraz zrobimy router w VM1.

```
root@debian1:/home/deb1# ip addr add 192.168.200.11/24 dev lo
root@debian1:/home/deb1# ip -br addr
lo               UNKNOWN        127.0.0.1/8 192.168.200.11/24 ::1/128
ens18            UP             192.[redacted]/24 fe[redacted]/64
ens19            UP             192.168.100.11/24 [redacted]/64
wg0              UNKNOWN        10.10.0.1/32
root@debian1:/home/deb1# _
```

Dodaliśmy adres innej podsieci do interfejsu “lo” (NIE usuwamy loopbacka 127.0.0.1).

Ustawiamy przekazywanie pakietów w jądrze, dzięki temu nasz host może działać jako router i przekazywać ruch między interfejsami.

```
root@debian1:/home/deb1# sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@debian1:/home/deb1# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
root@debian1:/home/deb1#
```

Aby ustawienie przetrwało restart wpisujemy do /etc/sysctl.conf

```
 GNU nano 8.4                                                    /etc/sysctl.conf
net.ipv4.ip_forward=1
```

Zaktualizujemy konfiguracje wireguarda na pozostałych maszynach.

```
 GNU nano 8.4                                                    /e
# Name VM3
[Interface]
PrivateKey = GH
Address = 10.10.0.3
ListenPort = 51820

[Peer]
# VM1
PublicKey = i2
Endpoint = 192.168.100.11:51820
AllowedIPs = 10.10.0.1, 192.168.200.11
PersistentKeepalive = 25

[Peer]
# VM2
PublicKey = c1yc
Endpoint = 192.168.100.12:51820
AllowedIPs = 10.10.0.2
Persi
```

```
e "systemctl status wg-quick@wg.service" and "journalctl -xeu wg-quick@wg.service" for details.
root@deb3min:/home/debian3min# systemctl restart wg-quick@wg0

root@deb3min:/home/debian3min#
root@deb3min:/home/debian3min# systemctl status wg-quick@wg0
● wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0
     Loaded: loaded (/usr/lib/systemd/system/wg-quick@.service; disabled; preset: enabled)
     Active: active (exited) since Sat 2026-10-24 15:03:37 EDT; 8s ago
 Invocation: 97fdc5f58f3a41a2a5cd8a362cc95daa
       Docs: man:wg-quick(8)
             man:wg(8)
             https://www.wireguard.com/
             https://www.wireguard.com/quickstart/
             https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
             https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8
    Process: 1938 ExecStart=/usr/bin/wg-quick up wg0 (code=exited, status=0/SUCCESS)
   Main PID: 1938 (code=exited, status=0/SUCCESS)
   Mem peak: 2.1M
        CPU: 58ms

Oct 24 15:03:37 deb3min systemd[1]: Starting wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0...
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] ip link add wg0 type wireguard
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] wg setconf wg0 /dev/fd/63
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] ip -4 address add 10.10.0.3 dev wg0
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] ip link set mtu 1420 up dev wg0
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] ip -4 route add 192.168.200.11/32 dev wg0
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] ip -4 route add 10.10.0.2/32 dev wg0
Oct 24 15:03:37 deb3min wg-quick[1938]: [#] ip -4 route add 10.10.0.1/32 dev wg0
Oct 24 15:03:37 deb3min systemd[1]: Finished wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0.
root@deb3min:/home/debian3min#
```

W węźle VM1 dodajemy nowy dozwolony adres IP oraz restartujemy usługę wireguarda na maszynach. NIE DODAJEMY TEGO ADRESU DO INTERFEJSU!

Sprawdzmy jak działa nasza nowa konfiguracja, czy nasz ruch do nowej podsieci dociera i czy idzie przez tunel VPN.

Włączamy na VM1 obserwowanie pakietów w tunelu (wcześniej sprawdzaliśmy tylko czy handshake pracuje i jest transfer, teraz robimy mocniejszy test)

```
root@debian1:/home/deb1# tcpdump -ni wg0 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wg0, link-type RAW (Raw IP), snapshot length 262144 bytes
_
```

Pingujemy z VM2.

```
root@deb2min:/home/debian2min# ping -c 3 192.168.200.11
PING 192.168.200.11 (192.168.200.11) 56(84) bytes of data.
64 bytes from 192.168.200.11: icmp_seq=1 ttl=64 time=0.727 ms
64 bytes from 192.168.200.11: icmp_seq=2 ttl=64 time=0.586 ms
64 bytes from 192.168.200.11: icmp_seq=3 ttl=64 time=0.720 ms

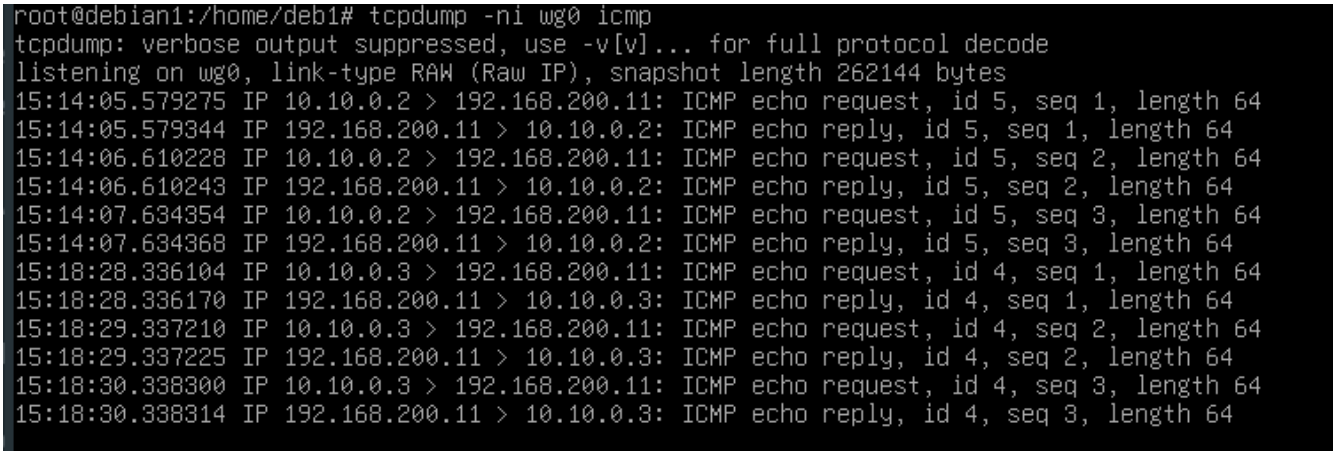
--- 192.168.200.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2055ms
rtt min/avg/max/mdev = 0.586/0.677/0.727/0.064 ms
root@deb2min:/home/debian2min#
```

Sprawdzamy co pojawiło się w tunelu.

```
root@debian1:/home/deb1# tcpdump -ni wg0 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wg0, link-type RAW (Raw IP), snapshot length 262144 bytes
15:14:05.579275 IP 10.10.0.2 > 192.168.200.11: ICMP echo request, id 5, seq 1, length 64
15:14:05.579344 IP 192.168.200.11 > 10.10.0.2: ICMP echo reply, id 5, seq 1, length 64
15:14:06.610228 IP 10.10.0.2 > 192.168.200.11: ICMP echo request, id 5, seq 2, length 64
15:14:06.610243 IP 192.168.200.11 > 10.10.0.2: ICMP echo reply, id 5, seq 2, length 64
15:14:07.634354 IP 10.10.0.2 > 192.168.200.11: ICMP echo request, id 5, seq 3, length 64
15:14:07.634368 IP 192.168.200.11 > 10.10.0.2: ICMP echo reply, id 5, seq 3, length 64
_
```

Jak widać nasz ping przeszedł przez tunel naszego VPNa.

Sprawdzmy to samo z wykorzystaniem VM3.



Jak widać ping do innej podsieci również przeszedł przez tunel naszego VPNa.

4. Podsumowanie

Zadanie nauczyło mnie konfiguracji VPNa z wykorzystaniem Wireguarda.
Przygotowaliśmy środowisko w Proxmoxie pod nasze 3 maszyny wirtualne, na których następnie zrobiliśmy adresację IP.
Skonfigurowaliśmy VPN mesha i sprawdziliśmy czy ruch idzie przez tunel.
Włączyliśmy usługę SSH na jednym hoście ii zastosowaliśmy reguły firewall z wykorzystaniem iptables, w celu zablokowania ruchu spoza VPNa, przeprowadziliśmy test czy przez LAN i VPN usługa jest osiągalna.
Dodaliśmy nową podsieć i ustawiliśmy jednego hosta w tryb routera, po czym skonfigurowaliśmy routing przez VPNa aby można było się komunikować z innymi podsieciami, na koniec przeprowadziliśmy test w którym sprawdziliśmy czy ruch do innej podsieci przechodzi przez tunel VPN.

Środowisko wykorzystane do realizacji zadania
-Proxmox Virtual Environment 9.2.21
-Debian 13.7.0
-Wireguard 1.0.20210914

Źródła:

https://www.zenarmor.com/docs/network-security-tutorials/how-to-configure-wireguard-mesh-vpn

https://docs.procustodibus.com/guide/peers/private-key/

https://www.wireguard.com/quickstart/#quick-start

https://www.debian.org/distrib/

https://wiki.archlinux.org/title/Iptables#Editing_rules

https://linux.die.net/man/8/iptables

Autor: Bartosz Marcińczyk